

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Mukesh Raj L	Reg. No.:	192571036
Date:		Duration:	60 minutes

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

- 1. Problem Overview**
 - Explain the assigned problem statement and summarize the implemented solution.
 - Describe the project objectives and key functionalities.
- 2. Repository Organization**
 - Evaluate the repository structure, folder organization, and file naming conventions.
 - Justify how the repository layout supports maintainability and collaboration.
- 3. Code Quality Review**
 - Review the source code for readability, modularity, coding standards, and documentation.
 - Identify strengths and areas for improvement.
- 4. Version Control Evaluation**
 - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
 - Explain how version control supports collaborative software development.
- 5. Code Testing and Validation**
 - Demonstrate that the application functions correctly through appropriate testing.
 - Present test cases, execution results, and screenshots as evidence.
- 6. Repository Documentation**

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

	organized.			
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. PROBLEM OVERVIEW

1.1 Problem Statement

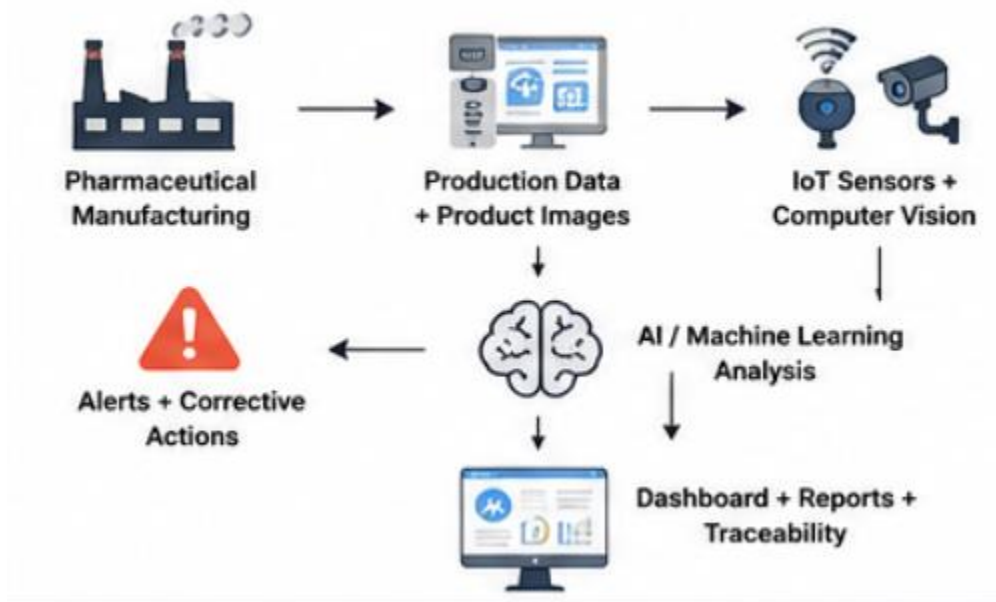
Pharmaceutical manufacturing requires strict quality assurance to ensure that medicines are safe, effective, and produced according to required quality standards. Traditional quality inspection processes often depend on manual inspection, periodic monitoring, and documentation. These processes can be time-consuming and may result in delayed identification of manufacturing defects.

The proposed **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform** addresses these challenges by combining Artificial Intelligence, Computer Vision, IoT, Machine Learning, and cloud-based technologies. The system is designed to continuously monitor manufacturing parameters, detect product defects, predict possible quality deviations, maintain batch traceability, and support quality and compliance reporting.

The platform provides a centralized environment where production and quality information can be monitored through a dashboard. AI-based inspection can identify visible product defects, while Machine Learning can analyze production data to identify possible quality deviations.

The system aims to reduce manufacturing defects and product wastage while improving quality monitoring, traceability, operational efficiency, and decision-making.

Figure 1: Problem Overview



Suggested diagram:



1.2 Implemented Solution

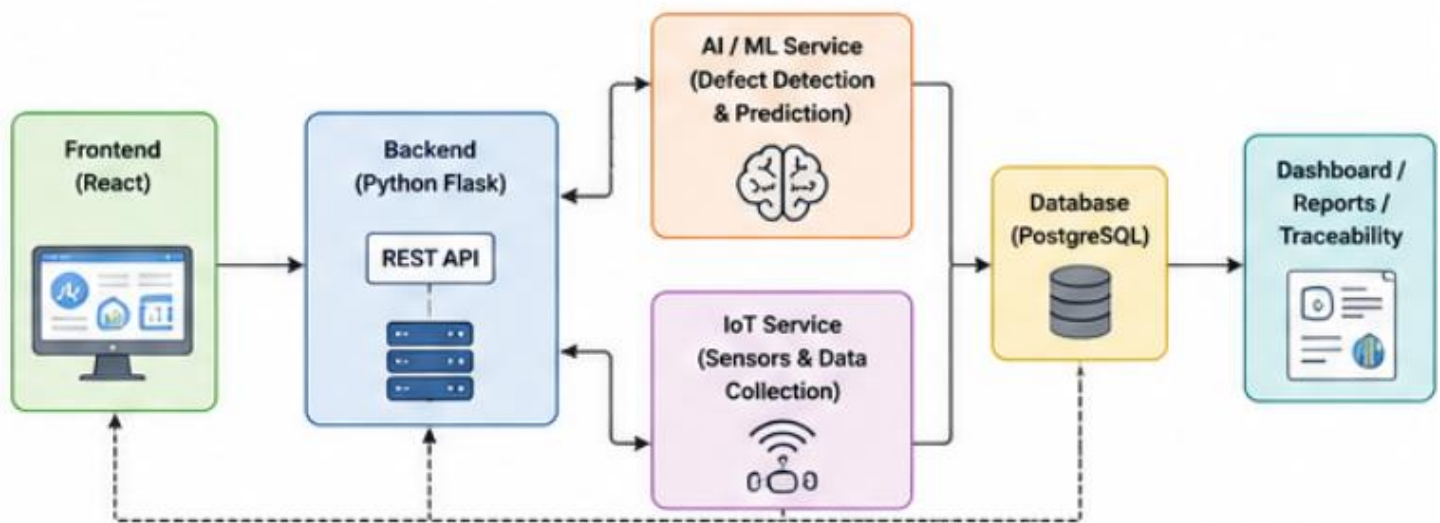
The proposed solution is organized as a modular software platform consisting of frontend, backend, AI/ML, IoT, database, and testing components.

The frontend provides the user interface through which authorized users can monitor production and quality information. The backend manages APIs, business logic, authentication, and communication between different system components.

The AI module performs defect detection and quality prediction. The IoT module collects manufacturing parameters from connected sensors. The database stores production information, inspection results, batch records, and quality-related data.

The system follows a centralized workflow in which production data is collected, analyzed, stored, and presented to authorized users through the dashboard.

Figure 2: Implemented Solution Overview



Frontend → Backend → AI/ML + IoT → Database → Dashboard/Reports

1.3 Project Objectives

The major objectives of the project are:

1. To continuously monitor pharmaceutical manufacturing processes.
2. To collect production parameters using IoT sensors.
3. To detect product defects using Artificial Intelligence and Computer Vision.
4. To predict possible quality deviations using Machine Learning.
5. To maintain pharmaceutical batch traceability.
6. To generate quality and compliance reports.
7. To provide alerts for abnormal production conditions.
8. To recommend corrective actions for quality deviations.
9. To reduce product wastage and manufacturing losses.
10. To improve manufacturing quality and operational efficiency.

1.4 Key Functionalities

The major functionalities of the platform include:

Real-Time Monitoring

The system monitors important production parameters and provides current manufacturing information.

AI-Based Defect Detection

Computer Vision can be used to inspect pharmaceutical products and identify visible defects.

Quality Prediction

Machine Learning can analyze production information and predict possible quality deviations.

Batch Traceability

The system maintains information related to production batches to support tracking and quality analysis.

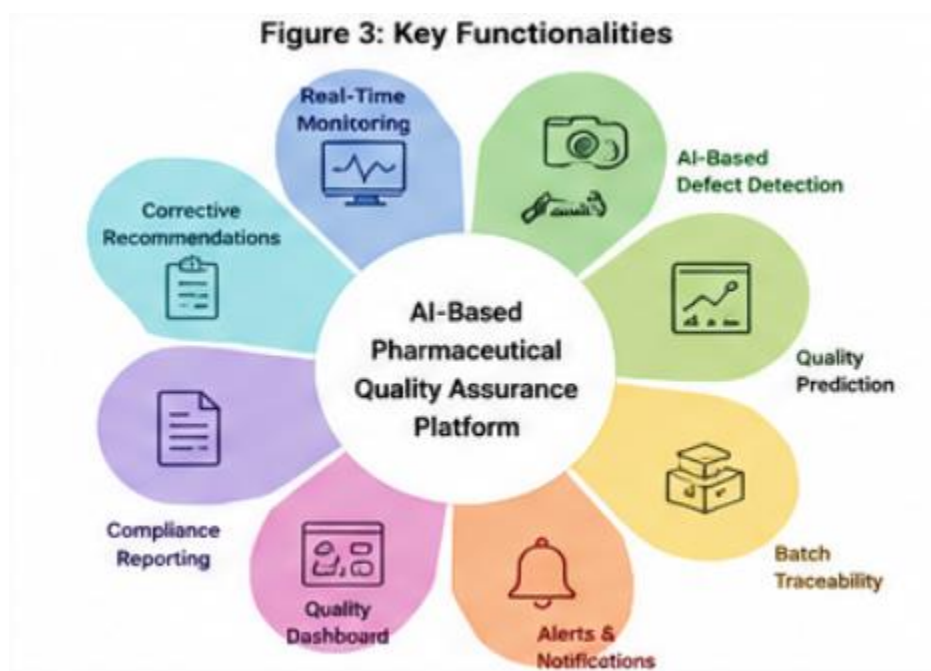
Alerts and Notifications

The platform can notify quality personnel when abnormal conditions or defects are identified.

Quality Dashboard

The dashboard provides a centralized view of production and quality information.

Compliance Reporting

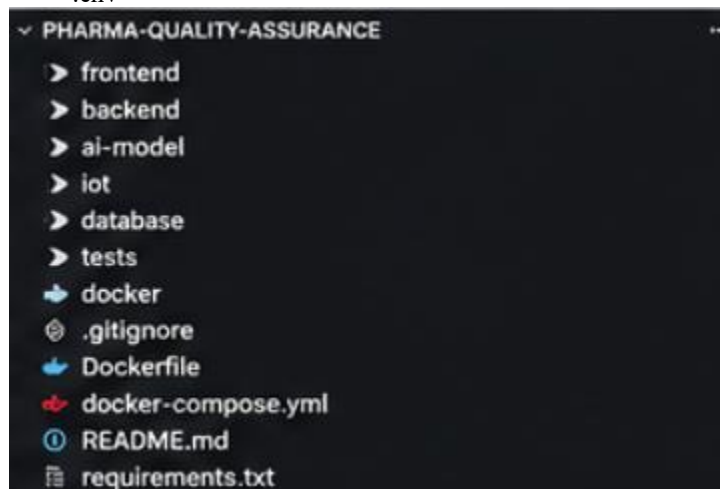
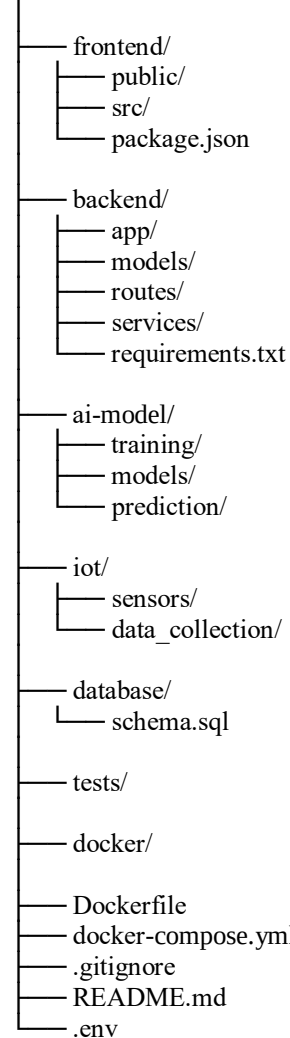


2. REPOSITORY ORGANIZATION

The repository is organized into separate folders for the major components of the pharmaceutical quality assurance platform. This modular organization improves maintainability and allows developers to work on different components independently.

2.1 Repository Structure

pharma-quality-assurance/



2.2 Folder Evaluation

Folder/File	Purpose	Evaluation
frontend/	User interface and dashboard	Clearly separates presentation layer
backend/	APIs and business logic	Supports modular backend development
ai-model/	AI/ML functionality	Separates AI functionality from application logic
iot/	Sensor and production data	Provides dedicated IoT functionality
database/	Database scripts	Centralizes database-related files
tests/	Test cases	Separates testing from application code
Dockerfile	Container configuration	Supports containerized deployment
docker-compose.yml	Multi-container configuration	Simplifies service management
.gitignore	Excludes unnecessary files	Helps maintain a clean repository
README.md	Project documentation	Helps users understand and run the project

2.3 File Naming Conventions

Consistent file naming improves readability and makes the repository easier to navigate.

The following conventions are recommended:

- Use meaningful names.
- Avoid unnecessary abbreviations.
- Use lowercase names where appropriate.
- Use consistent naming for related files.
- Use descriptive names for modules and services.

Examples:

```
quality_service.py
batch_controller.py
defect_detector.py
production_data.py
quality_dashboard.jsx
```

Poor naming should be avoided:

```
abc.py
test1.py
newfile.py
finalfinal.py
```

2.4 Maintainability and Collaboration

The repository structure supports maintainability because each major functionality is separated into its own module. Developers can modify the AI module without unnecessarily changing the frontend or database components.

The structure also supports collaboration because different developers can work on different modules using separate Git branches. This reduces the possibility of unrelated changes interfering with each other.

3. CODE QUALITY REVIEW

3.1 Readability

The source code should be written using clear variable names, meaningful function names, proper indentation, and logical organization.

For example:

```
def detect_product_defect(image):  
    result = model.predict(image)  
    return result
```

The function name `detect_product_defect()` clearly explains its purpose.

A less readable approach would be:

```
def d(x):  
    y = m.predict(x)  
    return y
```

The first approach is preferable because another developer can understand the function without examining every line.

3.2 Modularity

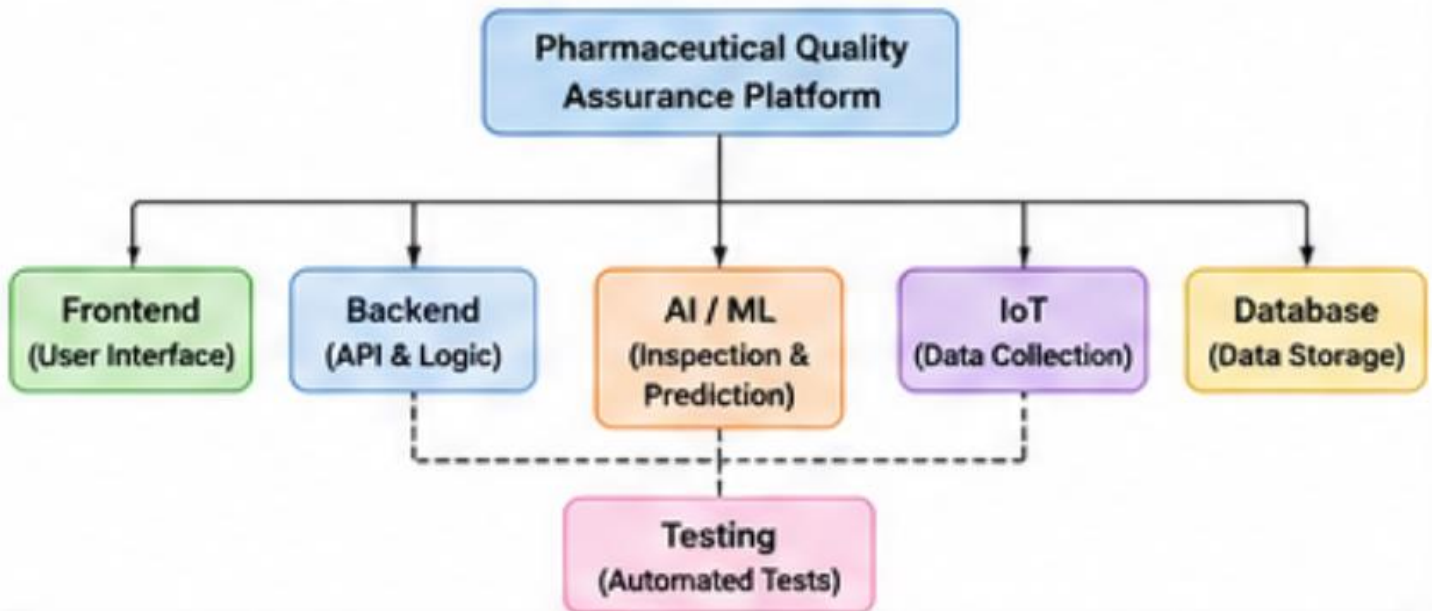
The project follows a modular approach by separating major functionalities such as:

- User interface
- Backend APIs
- AI/ML processing
- IoT data collection
- Database management
- Testing

Modularity allows individual components to be developed and tested independently.

For example, the defect detection functionality can be maintained separately from the batch management functionality.

Figure 4: Modular Organization of Components



3.3 Coding Standards

The following coding practices should be followed:

1. Use meaningful variable and function names.
2. Maintain consistent indentation.
3. Avoid unnecessary duplicate code.
4. Use comments where complex logic requires explanation.
5. Separate functions according to responsibility.
6. Avoid hard-coded sensitive information.
7. Maintain consistent file naming.
8. Handle errors appropriately.
9. Keep functions reasonably small.
10. Follow the conventions of the programming language being used.

3.4 Documentation in Code

Comments and documentation should explain complex logic rather than describing obvious statements.

Example:

```
# Analyze the captured product image# and identify possible manufacturing defects.def
detect_product_defect(image):
    return model.predict(image)
```

Documentation is especially important for AI and quality-related modules because future developers need to understand how the prediction and inspection processes work.

3.5 Strengths Identified

The major strengths of the proposed code organization are:

- Modular project structure.
- Separation of major application components.
- Meaningful naming of important modules.
- Dedicated AI/ML component.
- Dedicated testing directory.
- Docker configuration for deployment.
- Git-based version control.
- Potential for independent module development.

3.6 Areas for Improvement

The following improvements can strengthen code quality:

- Increase automated test coverage.
- Add more detailed documentation.
- Improve exception handling.
- Remove duplicate code.
- Use configuration files instead of hard-coded values.
- Apply consistent formatting.
- Add logging for important operations.
- Use static code analysis tools.
- Perform security scanning.
- Improve AI model validation and monitoring.

Backend code(Python)

```

1  @app.route('/api/quality/defect-detect', methods=['POST'])
2  def detect_defect():
3      file = request.files['image']
4      img = Image.open(file.stream)
5      result = model.predict(img)
6      return jsonify({'result': result})

```

AI/ML Code(Python)

```

def predict_defect(image):
    img = image.resize((224, 224))
    img_array = img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)
    prediction = model.predict(img_array)
    label = class_names[np.argmax(prediction)]
    confidence = float(np.max(prediction))
    return label, confidence

```

Frontend Code(React JSX)

```
<div className="dashboard">
  <h2>Quality Dashboard</h2>
  <div className="cards">
    <Card title="Total Batches" value={batches} />
    <Card title="Defect Detected" value={defects} />
    <Card title="Quality Score" value={qualityScore} />
  </div>
</div>
```

The assignment's highest code-quality rubric expects evaluation of readability, modularity, coding standards, documentation, and maintainability with clear observations.

4. VERSION CONTROL EVALUATION

4.1 Commit History

Git commit history provides a record of changes made to the project.

Example meaningful commits:

- Initial project setup
- Add pharmaceutical quality dashboard
- Implement backend API
- Add AI defect detection module
- Add IoT monitoring
- Implement batch traceability
- Add compliance reporting
- Add Docker configuration
- Update project documentation
- Fix quality monitoring API

Meaningful commits make it easier to understand the development history.

Git Commit History

```
$ git log --oneline
f7a9c2e Update project documentation
d3b7a91 Fix quality monitoring API
a1c3e45 Add Docker configuration
9e7b2d1 Implement compliance reporting
6b7f2a3 Add batch traceability module
4d8c1f9 Add IoT monitoring service
2c4d8b7 Add AI defect detection module
8a1d3e2 Implement backend API
3f2b1c0 Add pharmaceutical quality dashboard
1e2d3f4 Initial project setup
```

4.2 Commit Message Evaluation

Good commit:

Add AI defect detection module

Poor commit:

changes

The first message clearly communicates the purpose of the change, while the second provides insufficient information.

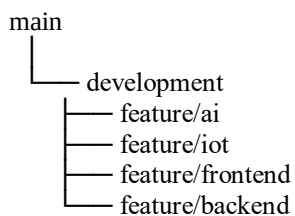
Evaluation

The repository should use:

- Short but descriptive commit messages.
- One logical change per commit.
- Consistent commit wording.
- Avoidance of meaningless messages.

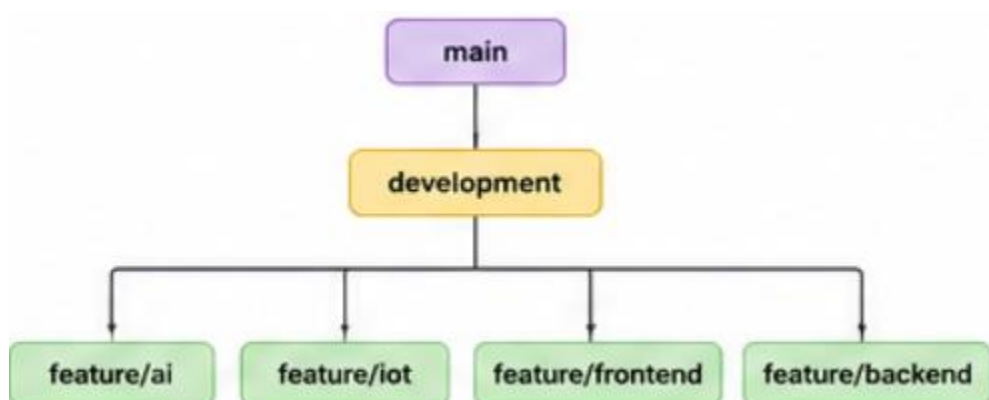
4.3 Branching Strategy

The project can use:



The main branch represents the stable version, while development is used for integration. Feature branches isolate individual functionalities.

Git Branching Strategy



4.4 Repository Maintenance

Repository maintenance includes:

- Removing unnecessary files.
- Maintaining .gitignore.
- Keeping documentation updated.
- Using meaningful commits.
- Deleting obsolete branches.
- Reviewing pull requests.
- Avoiding sensitive information in Git.
- Keeping dependencies updated.

4.5 Collaboration Support

Git supports collaborative software development by allowing developers to work on separate branches and merge their completed work after review.

For this project, one developer could work on AI functionality while another works on the frontend or IoT module. These changes can later be integrated into the development branch.

The assignment specifically asks for commit history, branching strategy, commit messages, repository maintenance, and explanation of collaborative development.

5. CODE TESTING AND VALIDATION

Testing is performed to verify that the application's major functionalities work as expected. The assignment specifically requires test cases, execution results, and screenshots as evidence.

5.1 Test Case Table

Test ID	Test Case	Expected Result	Status
TC01	Open application	Application loads successfully	Pass/Fail
TC02	User login	Authorized user can log in	Pass/Fail
TC03	Dashboard loading	Dashboard displays correctly	Pass/Fail
TC04	Production data	Production parameters are displayed	Pass/Fail
TC05	Defect detection	Defect analysis produces a result	Pass/Fail
TC06	Quality prediction	Prediction result is generated	Pass/Fail
TC07	Batch tracking	Batch information is retrieved correctly	Pass/Fail
TC08	Alert generation	Alert appears for abnormal condition	Pass/Fail
TC09	Report generation	Quality report is generated	Pass/Fail
TC10	Database connection	Data is stored/retrieved correctly	Pass/Fail

Important: Change Pass/Fail to the **actual result** after you test your application.

5.2 Functional Testing

Functional testing verifies whether each major feature performs its intended function.

For example, when a user opens the quality dashboard, the expected result is that the dashboard loads and displays the available quality and production information.

Similarly, when product images are provided to the defect detection module, the system should return the corresponding analysis result.

5.3 Integration Testing

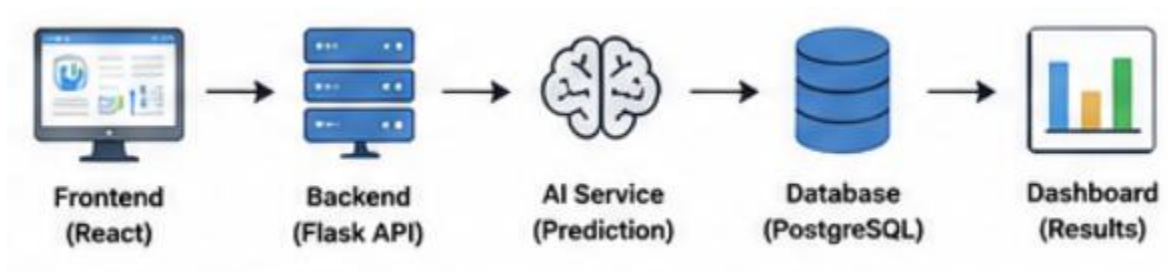
Integration testing verifies communication between different components.

Examples include:

Frontend → Backend
Backend → AI Service
Backend → Database
IoT → Backend
Backend → Dashboard

Successful communication indicates that the individual components can work together.

Component Integration Testing Flow



5.4 Docker Testing

The containerized application can be tested using:

```
docker compose up -d
```

Then:

```
docker ps
```

The running containers should be displayed.

Docker Compose Services Running

```
$ docker compose up -d
[+] Running 5/5
 ✓ Container backend      Started
 ✓ Container frontend     Started
 ✓ Container db            Started
 ✓ Container ai-service   Started
 ✓ Container iot-service  Started
```

Output

```
$ docker ps

CONTAINER ID   IMAGE          STATUS               PORTS
a1b2c3d4e5f6   pharma-frontend Up 3 minutes        0.0.0.0:3000->3000/tcp
b1c2d3e4f5g6   pharma-backend Up 3 minutes        0.0.0.0:5000->5000/tcp
c1d2e3f4g5h6   postgres:14   Up 3 minutes        0.0.0.0:5432->5432/tcp
d1e2f3g4h5i6   ai-service    Up 3 minutes        0.0.0.0:8000->8000/tcp
e1f2g3h4i5j6   iot-service   Up 3 minutes        0.0.0.0:9000->9000/tcp
```

Application Running in Browser



Test Execution Result

```
$ pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest-7.4.3
collected 10 items

.....
===== 10 passed in 2.34s =====
[100%]
```

5.5 Testing Evaluation

The testing process should verify:

- Functional correctness.
- Service communication.

- Database connectivity.
- AI service operation.
- IoT data processing.
- User interface functionality.
- Container execution.
- Error handling.

The highest rubric level requires comprehensive testing with appropriate test cases, execution results, and supporting evidence.

6. REPOSITORY DOCUMENTATION

6.1 README Evaluation

The README.md file should provide a clear introduction to the project and explain how users can install, configure, and run it.

A good README should contain:

1. Project title.
2. Project description.
3. Problem statement.
4. Objectives.
5. Features.
6. Technology stack.
7. Repository structure.
8. Installation instructions.
9. Configuration instructions.
10. Running instructions.
11. Testing instructions.
12. Docker instructions.
13. Usage guide.
14. Future enhancements.

6.2 Installation Instructions

The README should explain the prerequisites and installation process.

Example:

1. Clone the repository.
2. Install required dependencies.
3. Configure environment variables.
4. Configure the database.
5. Build Docker images.
6. Start the services.
7. Open the application.

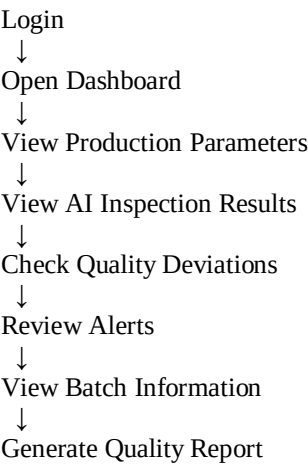
Docker-based execution can be documented as:

```
docker compose build
docker compose up -d
```

6.3 Usage Guide

The usage guide should explain how users interact with the application.

Example workflow:



6.4 Dependencies

The documentation should clearly identify the major dependencies used by the project.

Example:

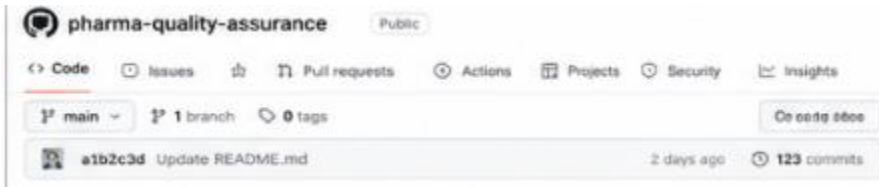
Component	Possible Technology
Frontend	React / HTML / CSS / JavaScript
Backend	Python / Flask / FastAPI
AI/ML	Python / Machine Learning libraries
Database	PostgreSQL
IoT	Sensor/IoT framework
Version Control	Git/GitHub
Containerization	Docker/Docker Compose

6.5 Documentation Improvements

The documentation can be improved by:

- Adding screenshots of the application.
- Providing step-by-step installation instructions.
- Adding API documentation.
- Explaining environment variables.
- Providing troubleshooting information.
- Adding testing instructions.
- Documenting Docker commands.
- Providing contact/project information where appropriate.

README.md in GitHub



7. CODE REVIEW FINDINGS AND RECOMMENDATIONS

7.1 Overall Code Review Findings

The review of the proposed pharmaceutical quality assurance platform indicates that a modular architecture and organized repository provide a suitable foundation for further development.

The separation of frontend, backend, AI/ML, IoT, database, and testing components improves maintainability and allows individual modules to evolve independently.

The use of Git provides version tracking, branching, and collaboration capabilities, while Docker supports consistent application environments.

However, the project can be further strengthened through improved automated testing, documentation, security practices, logging, and code-quality analysis.

7.2 Major Strengths

1. Modular Organization

The project separates major functionalities into different components, improving maintainability.

2. Clear Functional Separation

Frontend, backend, AI, IoT, and database responsibilities are separated.

3. Version Control

Git provides a structured mechanism for tracking development changes.

4. Containerization

Docker provides a consistent environment for application deployment.

5. AI Integration

The platform provides opportunities for automated quality inspection and prediction.

6. Scalability

The modular structure allows additional services and manufacturing capabilities to be added later.

7.3 Identified Weaknesses

The following areas should be considered for improvement:

- Limited automated test coverage.
- Need for stronger exception handling.
- Need for better API documentation.
- Potential dependency management issues.
- Need for secure management of credentials.
- Need for continuous AI model evaluation.
- Need for stronger logging and monitoring.
- Need for more detailed README instructions.

7.4 Recommendations

Recommendation 1 – Improve Automated Testing

Develop unit, integration, and end-to-end tests to verify system functionality automatically.

Recommendation 2 – Implement CI/CD

A CI/CD pipeline can automatically:

```
Git Push
↓
Build
↓
Run Tests
↓
Code Quality Check
↓
Docker Build
↓
Deployment
```

Recommendation 3 – Improve Security

Sensitive credentials should not be stored directly in source code or committed to Git.

Use environment variables or secure secret-management mechanisms.

Recommendation 4 – Improve Documentation

The README should include complete installation, configuration, usage, testing, and Docker instructions.

Recommendation 5 – Add Code Quality Tools

Static analysis and formatting tools can help identify coding problems early.

Examples include:

- Code formatting.
- Linting.
- Static analysis.
- Dependency scanning.
- Security scanning.

Recommendation 6 – Improve Logging

The backend and AI services should maintain useful logs for:

- Errors.
- API requests.
- AI predictions.
- Sensor failures.
- Database problems.
- Authentication events.

Recommendation 7 – Improve AI Model Monitoring

The AI model should be evaluated regularly using appropriate validation data. Changes in production conditions should be monitored to identify potential model-performance degradation.

Recommendation 8 – Improve Testing Evidence

Every major feature should have corresponding test cases and screenshots showing the actual execution result.

7.5 Future Enhancements

Future versions of the system could include:

1. Advanced Computer Vision models.
2. Real-time AI inference at the manufacturing edge.
3. Predictive maintenance of production equipment.
4. Advanced analytics dashboards.
5. Cloud-based deployment.
6. Kubernetes-based container orchestration.
7. Automated CI/CD deployment.
8. Enhanced batch traceability.
9. Advanced anomaly detection.
10. Multi-facility manufacturing support.

7.6 Final Conclusion

The **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform** provides a structured approach to improving pharmaceutical manufacturing quality assurance through AI, IoT, Computer Vision, Machine Learning, and centralized software services.

The repository organization separates major application components and supports maintainability and collaborative development. Git provides version control, branching, and change tracking, while Docker supports consistent deployment environments.

The code review highlights strengths in modularity and organization while identifying opportunities for improvement in automated testing, documentation, security, logging, and continuous integration.

Overall, the project provides a suitable foundation for an intelligent pharmaceutical quality assurance platform, with future improvements capable of increasing its reliability, scalability, maintainability, and deployment efficiency.